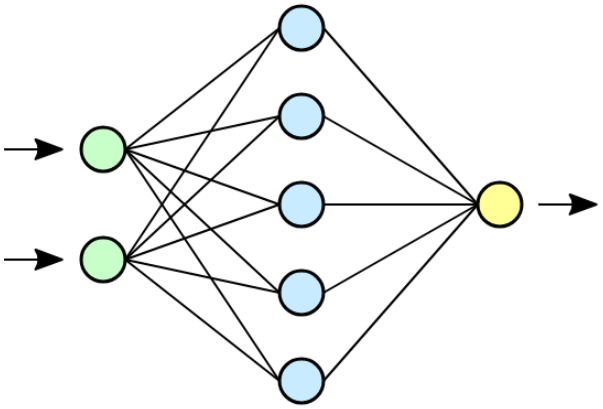
PPO (Proximal policy optimization) Algorithme documentation :

Le PPO algorithme est un programme majeur d'apprentissage par renforcement, il a été initialement développé pour entraîner le très connu ChatGPT par les équipes d'OpenAi. Cet algorithme se voulait plus simple d'utilisation et surtout plus stable que les autres algorithmes d'apprentissage par renforcement. Le PPO a déjà fait ses preuves notamment en environnement complexe comme lorsqu'il a gagné contre une équipe professionnelle de Dota2.

Il semblait donc non négociable de tester l'algorithme PPO et faire des recherches concernant notre agent à entraîné en environnement lunaire (Gymnasium). La principale différence entre les algorithmes DQN et double DQN que nous avions déjà implementait dans le projet c'est que le PPO est un algorithme dit On Policy ce qui signifie qu'il vas planifier une nombre N de steps avant de réajuster son réseau de neurone avec les donnée qu'il a récolté durant les N premiers step. Les algorithmes DQN et double DQN eux sont des algorithmes Off policy ce qui signifie qu'il vont se baser sur des données déjà existantes pour réajuster leur système de neurone.

Pour nous il est donc assez intéressant de comparer ces deux manière de procéder afin de potentiellement en désigner une comme étant plus adéquate à notre problème. Pour ce faire nous devrons dans un premier temps comprendre rapidement comment un réseau de neurones est constitué et comment il va prendre des décisions.

Les réseaux de neurones :



Voici un schéma simple de comment se présente un réseau de neurones, les rond sont appelés des nodes et ils sont reliés par des axes qui auront des poids associés. Les nodes verts sont les input en l'occurrence dans notre cas cela vas être le State qui comprend toute les valeurs d'environnement. Le node jaune vas etre le ou les choix possibles au finale apres le process des valeurs d'environnement. En traduction plus vulgaire on pourrait dire que l'on donne des informations à notre réseau pour qu'il choisisse le meilleur choix.

Évidemment au départ le réseau ne possède pas la science infuse et vas opéré de manière random et affiner sa reflection durant le training le but est d'affiner le decision making de notre réseau ce qui sous entend de modifier la valeur des poids sur les axes reliant les nodes les uns au autre et cela vas directement impacter les probabilités que le programme

choisissent au finale telle ou telle options. Pour ce faire, le programme distribué des récompenses (rewards) en fonction de si l'action est jugée bénéfique ou non.

Le fonctionnement de l'algorithmes PPO :

L'algorithme PPO est comme on l'avait dit un algorithme on Policy et aussi il opère avec le Actor - Critic method ce qui veut dire que il y a deux réseau de neurone, un pour le côté Actor de la method qui vas se charger du decision making et un autre réseau pour le côté Critic qui vas lui prédire un score à atteindre pour valider ou non les décision de l'actor. Pour le PPO il est important de préciser que les deux réseaux sont indépendants et bien qu'il prenne le même input (le State) il ne doivent pas forcément être identiques.

Ensuite il faut également une batch memory qui permet de stocker les valeur relative au choix qui ont été fait pour ajuster plus tard (au bout de N step) les réseaux de neurone respectif.

Le calcule des Reward est très important et va être utilisé dans le calcule de l'avantage. L'avantage a pour objectif de définir si la décision prise par l'algorithme était une bonne décision ou non.

A = Advantage
S = State
M = move chosen
V(S) = Critic value
Q(S, M) = Reward (Rt)

A(S, M) = Q(S, M) - V(S)

Dans ce calcul, si la valeur d'advantage est supérieur à 0 le choix vas être considéré comme correct alors qu'à l'inverse si l'advantage est inférieur à 0 le choix vas être considéré comme mauvais. En principe le V(S) à la prochaine séquence se rapprocherait de notre Q(S, M) afin d'affiner ses prédictions.

Passons maintenant au loss, les loss value vont être calculés de différente manière selon si on parle du réseau Actor ou du réseau Critic. L'objectif principal de ces valeurs va être de réajuster les poids dans le réseau.
Pour le réseau Critic on fait une sorte de moyenne (pour simplifier) ce qui va permettre au réseau de Critic de prédire des valeurs plus proches que ce que l'on peut réellement rencontrer.
Pour le réseau Actor on va procéder à un calcul nommé LClip. Voici comment se présente le calcule : Lclip(0) = Et [min(rt(0) * At, clip(rt(0), 1 - eps, 1 + eps) * At)]

Le concept de ce calcule vas etre de choisir la valeur la plus faible entre a et b : min(a, b). Pour ce faire, voyons en quoi consiste chacune de ces valeurs. La valeur ici appeler A est calculé de cette manière :

rt(0) = (probabilité du choix actuelle / ancienne probabilité du choix) * At*
*At correspond à l'advantage

La valeur B quant à elle vas se présenter de cette manière :

(1 - eps <= rt <= 1 + eps) * At

rt étant le calcule de la valeur À expliqué ci dessus. Comme on peut le remarquer, elle est comprise entre deux valeurs, 1 - eps et 1 + eps. Eps correspond à Epsilon qui est une valeur que nous définissons (Hyperparameter) cela veut dire que si le ratio calculé n'est pas compris entre ces deux valeurs alors il serait cap, cela permet de garder la main sur la valeur du loss et par conséquent de ne pas entraîner de changements trop radicaux au sein du réseau de neurone.

Notion intéressante : Pytorch est optimisé pour avoir un gradient descendant (minimiser les erreurs) or ce que nous cherchons à avoir avec l'algo PPO et le scénario lunar ship c'est un gradient ascendant qui vas nous permettre de reach les score les plus haut possible. Techniquement il suffit d'inverser le signe des loss.

Ces valeurs de Loss restent obscures si on ne comprend pas à quoi elles servent. Pour le comprendre il va falloir comprendre ce que l'on appelle ma backpropagation, cette notion est la notion qui permet de réajuster les poids entre les nodes d'un réseau de neurone, elle est la même pour tous les réseau de neurone existant. Pour bien comprendre cette notion il faut imaginer un programme qui pass sur chaque node du réseau et qui applique une formule en fonction des événements passés qui sont représentés par le loss, une autre valeur vas rentrer en compte le Learning rate qui est également une valeur que nous définissons (hyperparameter).

<u>Le PPO dans notre cas (Recherches et résultats) :</u>

Dans notre cas j'ai donc simplement reproduit l'algorithme en faisant en sorte de pouvoir aisément changer les paramètres d'apprentissage de l'algorithme. Le state donnée en input était l'état de la simulation retourné par gymnasium. Lors de mon premier jeux de test j'avais en hyper paramètres les valeurs suivante :

learning_rate: 3.0e-4 (0.0003)
gamma: 0.99
batch_size: 64 (nombre de donnée piocher dans les N steps)
timestep: 2048 (nombre d'étapes après laquelle notre algorithme apprend)
buffer_capacity: 30000
epsilon_decay: 0.2
ppo_epochs: 4 (nombre de fois que les données peuvent être relue par l'algorithme d'apprentissage)
hidden_size: 64 (taille de notre réseau de neurone)

episodes: 10000 (nombre d'épisode)

le résultat fut concluant sur la manière dont le programme "apprenais" et nous a donc poussé à approfondir le sujet et notamment a lancé de la même manière que pour lee DQN et le Double DQN un entraînement automatique qui changer d'hyper parametres afin de trouver la meilleur disposition.